

Seen

Assignment - 05

Tutorial sheet - 09

D5A (22BCN1068)

13/12/23

DATE: / /

PAGE NO.: 72

Section - A

1. Write the main Difference betⁿ. DFS and BFS.

Ans → The main difference betⁿ. DFS (Depth First search) and BFS (Breadth-First-search) is their order of node exploration. DFS goes deep into the graph or tree before exploring siblings, while BFS explores neighbors at the current depth level before moving to the next level.

BFS

- it uses the Queue data structure for finding the shortest path.
- BFS operates using the FIFO list.
- BFS is slower than DFS.
- it is not memory efficient.

DFS

- it uses the stack data structure for finding the shortest path.
- DFS operates using the LIFO list.
- DFS is faster than BFS.
- it is memory efficient.

2. Write the name of main operation on the graph.

Ans → Main operation on the graph are:

- (i) insertion of Nodes (Edges on graph)
- (ii) Deletion of Nodes (Edges into graph)
- (iii) Traversal of graph.
- (iv) Searching
- (v) storing or creating graphs.
- (vi) Find Shortest Path.

3. What is difference betⁿ. directed and undirected?

Ans Directed Graph: A directed graph, is a graph that is unidirectional in this, the edges have a specific direction and the edges have directions specified with them also a directed

Jan 2, 2024, 18:04 Can contain graph cycles.

- Undirected Graph: An undirected graph is a graph where the edges do not have a specific direction and it is bidirectional in nature. It does not have a parent-child relation concept as there is no particular direction.

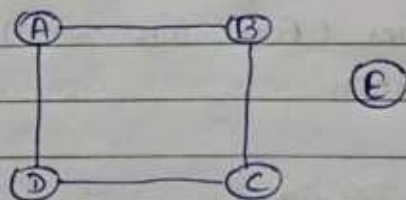
4. What do you mean by transpose of a graph?

Ans Transpose of a directed graph G is another directed graph on the same set of vertices with all of the edges reversed compared to the orientation of the corresponding edges in G . That is, if G contains edge (u, v) then its transpose of G edge is (v, u) .

5. What is isolated nodes on graph?

Ans A isolated node of a graph that has no edges, a vertex with degree zero. If a graph is not connected it consists of several components.

eg:



Here, Node E, is an isolated node.

Section - B

1. Write down difference betⁿ Prim's and Kruskal algo.

Ans Prim's Algo: Prim's algorithm is popular as a greedy algorithm that helps in discovering the smallest spanning tree for a weighted undirected graph that means, this algo tends to search the subgroup of the

edges that can construct a tree, and the complete poundage of all the edges in the tree should be minimal.

- **Kruskal Algo:** In the world of computer science, Kruskal's algorithm is used to discover the smallest spanning tree of a connected graph and the spanning forest of an undirected edge-weighted graph. Basically, it accepts a graph as input and discovers the subgroup of the edges of that graph.

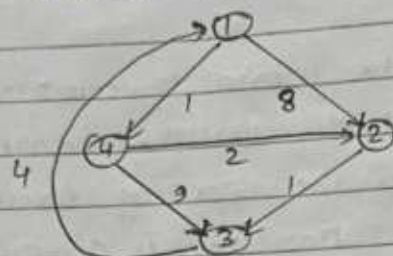
Prim's Algorithm

Kruskal's Algorithm

- | | |
|---|---|
| (i) This algorithm begin to construct the shortest spanning tree from any vertex in the graph. | This algo. begin to construct the shortest spanning tree from the vertex having the lowest weight in the graph. |
| (ii) To obtain the Minimum distance it traverses one node more than one time. | • it crosses one node only one time. |
| (iii) The time complexity of Prim's algorithm is $O(V^2)$. | • The time complexity of Kruskal's algorithm is $O(E \log V)$. |
| (iv) in Prim's algo., all the graph element must be connected. | • Kruskal's algo. may have disconnected graphs. |
| (v) it prefers list data structure. | • it prefers the heap data structure. |
| (vi) Prim's algo. runs faster in dense graphs. | • Kruskal's Algo. runs faster in sparse graph. |
| (vii) Applications of Prim's algorithm are, Travelling salesman Problem, Network for roads and Rail tracks connecting all the cities etc. | • Applications of Kruskal algo. are LAN connection, TV Network, etc. |

2. With an example discuss Warshall's algorithm.
 Ans All pair shortest path (Floyd Warshall's algorithm)

Step 1:



- Step 1: Construct an adjacency matrix

$$D^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 4 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

Step 2:

$$D^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 3 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

$$D^0[2,3] \quad D^0[2,1] + D^0[1,3] \\ 3 < \infty + \infty$$

$$D^0[2,4] \quad D^0[2,1] + D^0[1,4] \\ \infty \quad \infty + 1$$

Step 3:

$$D^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & 11 & 1 \\ \infty & 0 & 3 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 5 & 0 \end{bmatrix} \end{matrix}$$

Step 4:

$$D^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & 11 & 1 \\ 7 & 0 & 3 & 8 \\ 4 & 12 & 0 & 5 \\ 9 & 12 & 5 & 0 \end{bmatrix} \end{matrix}$$

	1	2	3	4
1	0	3	6	1
2	7	0	3	8
3	4	7	0	5
4	9	2	5	0

$$D^k[i, j] = \min \{ D^{k-1}[i, j], D^{k-1}[i, k] + D^{k-1}[k, j] \}$$

(3). Write the properties of DAG.

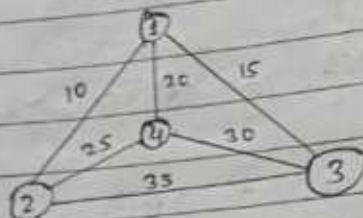
Ans Properties of DAG (Directed Acyclic Graph):-

DAG are a special type of graph with some unique and important properties that make them useful in various fields, including data science, scheduling, and machine learning.

- (i) Directed :- Edges in a DAG have a defined direction, indicating the flow of information or dependency between nodes.
- (ii) Acyclic :- DAGs do not contain cycles. This means there is no path that starts and ends at the same node, following only the direction of the edges.
- (iii) Reachability :- Any node in a DAG can be reached from any other node by following the directed paths.
- (iv) NO Self-loops :- Vertices can't have an edge that starts from same vertex.
- (v) Scalability :- DAGs can be easily scaled by adding new nodes and edges.
- (vi) Directed Acyclic Graphs have topological orderings defined.
- (vii) DAG is an efficient method for identifying common sub-expressions.

Section - C (3x11=33)

1. Solve the following problem with travelling sales person



Ans Conditions of Travelling Salesman Problem:-

- (i) No place is to be visited twice before the tour of all places is completed.
- (ii) Going from 2 place to 2 place is not allowed.

Now, we construct the adjacency matrix of the above graph:

	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0

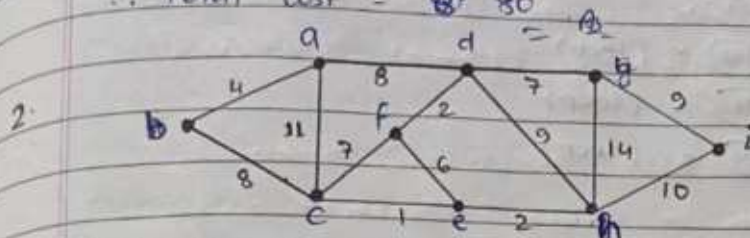
- (i) Now, salesman travelling from $1 \rightarrow 2$,
it cover distance = 10
- (ii) then, salesman traveling from $2 \rightarrow 4$,
distance = 25
- (iii) salesman travelling from $4 \rightarrow 3$,
distance = 30
- (iv) salesman travelling from $3 \rightarrow 1$,
return to its home, distance = 15

$\therefore$ total distance cover by the salesman is :-

This TSP tour in graph cycle = $1-2-4-3-1$.

Now, cost of tour is: $1 \rightarrow 2 + 2 \rightarrow 4 + 4 \rightarrow 3 + 3 \rightarrow 1$
 $= 10 + 25 + 30 + 15$
 $= 80$

$\therefore$ Total cost = ~~80~~ 80



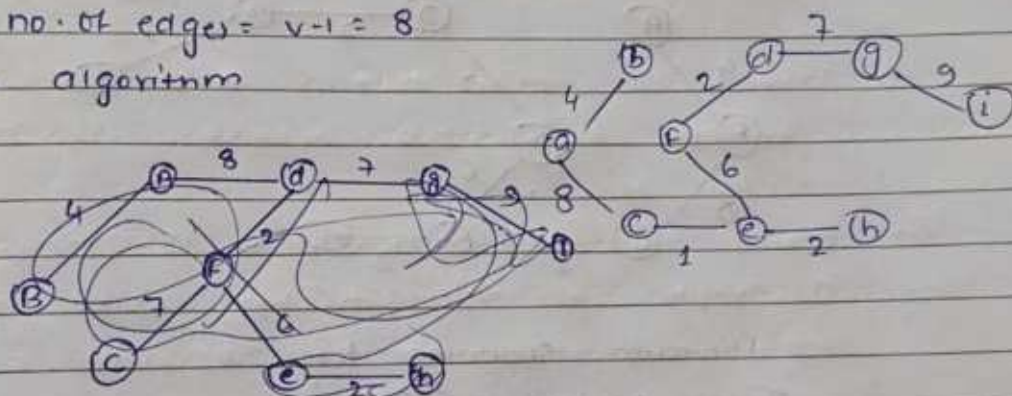
Using the above graph differentiate Prim and Kruskal Algorithm.

Ans

no. of vertex = $V = 9$

no. of edges = $V-1 = 8$

• Prim's algorithm



Minimum Spanning tree

Total weight = ~~105~~ $4+8+1+2+6+2+7+9 = 39$

• Kruskal Algorithm:

arrange weight in ascending order

CE = ① ✓

EH = ② ✓

FD = ② ✓

AB = ④ ✓

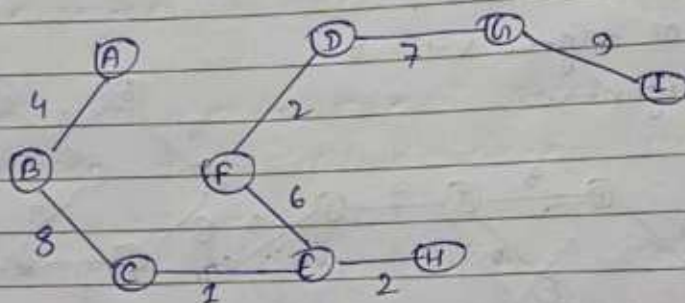
FE = ⑥ ✓

FC = ⑦ X (loop)

DG = ⑦ ✓

BC = ③ ✓
 AD = ⑧ ✗ (loop)
 DH = ⑨ ✗ (loop)
 GI = ⑨ ✓
 HI = ⑩ ✗ (loop)
 AC = ⑪ ✗ (loop)
 GH = ⑭ ✗ (loop)

Note, according to this order, we join vertex to form min-spanning tree



Minimum spanning tree
Total weight = 39

3. Explain any algorithm for all pairs shortest path problem.

→ Dijkstra's Algorithm :- Find shortest path from source node to all other nodes in a weighted graph by iteratively selecting the node with the smallest tentative distance to its neighbours.

→ Bellman - Ford Algorithm :- it is a single source shortest path algorithm that determines the shortest path between given source vertex and every other vertex in a graph.

→ Floyd-Warshall Algorithm :- The Floyd-Warshall Algo. is for solving all pair of shortest path problems. the problem is to find the shortest distance b/w every pair of vertices in a given edge weighted directed graph.

→ Breadth-First Search (BFS) for Shortest Path Algorithm :

BFS is a great shortest path algo. for all graphs, the path found by breadth first search to any node is the shortest path to that node, the path contains the smallest numbers of edges in unweighted graphs.

→ Depth-First Search for Shortest Path Algorithm :-

DFS Algo. recursively explores the adjacent nodes until it reaches to the depth, where no more valid recursive are possible.

Assignment-05
DSA
(Tutorial sheet - 10)

14/12/23

DATE: / /

PAGE NO.: 81

Seen

Section - A

1. What do you mean by greedy?

Ans. It means making locally optimal choice at each step, with hope that it will lead to a globally optimal solution.

2. What is difference betn graph and tree?

Ans. main difference is there structure.

A graph is a collection of nodes connected by edges, where edges can go in any direction and can have no of connection.

Tree is a specific type of graph that has ~~horizon~~ hierarchical structure.

3. Write about the data structure of graph.

Ans. A graph is a data structure comprising vertices (nodes) and edges. vertices represent entities and edges denote relationship between entities. The structure can be represented using an adjacency matrix or an adjacency list. Graphs can be directed or undirected and may have weights on edges.

4. What do you mean DAG?

Ans. A DAG (Directed Acyclic Graph) is a graph data structure with directed edges, meaning each edge has a specific direction from one vertex to another. The critical property of a DAG is that it is acyclic, ensuring there are no cycles or loops in the graph.

Q. What do you mean by dynamic programming?
 Ans. It is a technique used to solve complex problem by breaking them down into smaller overlapping subproblems. It's like solving a big problem by solving series of smaller related problems and then combining their solutions.

Section-B

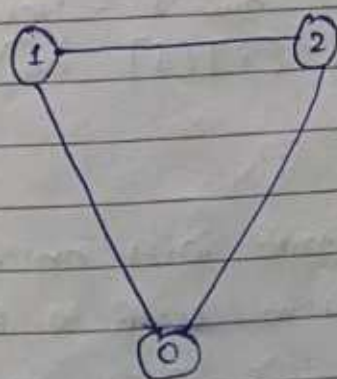
(1) Discuss How to represent graph storage using Adjacency matrix.

Ans. Adjacency matrix is 2D matrix that represents connection betⁿ nodes in graph.

• Representation of Undirected graph to matrix.

- if there is an edge from vertex i to j . mark $adjMat[i][j]$ as 1.
- if there is no edge from vertex i to j . Mark $adjMat[i][j]$ as 0.

→ initially, the entire matrix is initialized to 0. If there is an edge from source to destination, we insert 1 to both rows ($adjMat[source]$ and $adjMat[destination]$) because we can go either way.

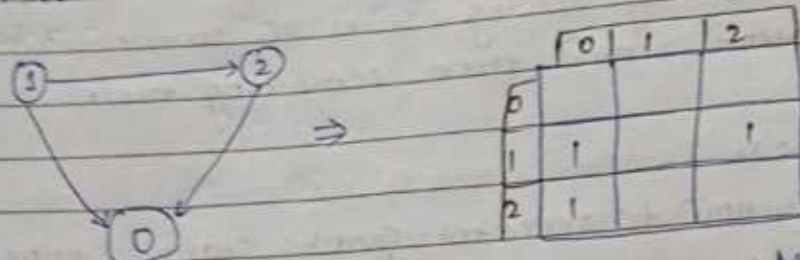


Undirected graph

	0	1	2
0		1	1
1	1		1
2	1	1	

Adjacency Matrix

- Representation of Directed Graph to Adjacency Matrix:-
- Initially, the entire matrix is initialized to 0. If there is an edge from source to destination, we insert 1 for that particular adjMat[destination].



Directed Graph

Adjacency Matrix

- Here How it works -
- Create a matrix -
matrix size is determined by no. of nodes in graph.
- initialize the matrix.
initially all the cells in matrix are set to 0 or false indicating no connection betⁿ nodes.
- Represent Connections -
For each edge in graph, you update corresponding cells in matrix. If there is edge b/w node i and j, you set the value of matrix [i][j] and [j][i] to 1 or true.

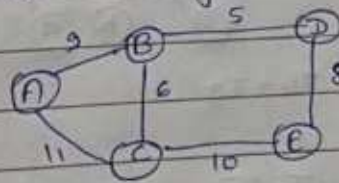
2. Explain Dijkstra's algorithm with an example.

Ans. It is a popular algorithm used to find Shortest Path between two nodes in graph.

Jan 2, 2024, 18:05 Procedure :-

- Start by assigning a tentative distance value to every node in graph. set initial node distance to 0 and all other to ∞ .
- Mark initial node as visited and set it as current node.
- For current node, consider all of its adjacent unvisited neighbours and calculate tentative distance through current node. Compare these calculated distances with current assigned value and update tentative distances if they are smaller.
- Once you have considered all unvisited neighbour of current node, it won't be checked again.
- If destination node has been marked visited or if smallest tentative distance among unvisited nodes is ∞ , then stop.
- Otherwise, select unvisited node with smallest distance as current node and go back to step 3.

example:



Starting from A, it would find shortest distⁿ to all nodes

Q 3. Construct DAG For

$$T_1 = a + b$$

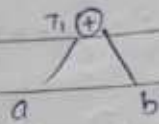
$$T_2 = a - b$$

$$T_3 = T_1 \times T_2$$

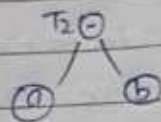
$$T_4 = T_1 - T_3$$

$$T_5 = T_4 + T_3$$

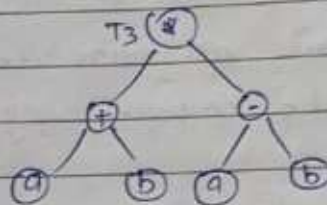
Ans Step 1: $T_1 = a + b$



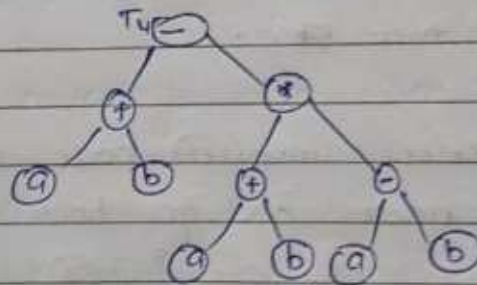
• Step 2: $T_2 = a - b$



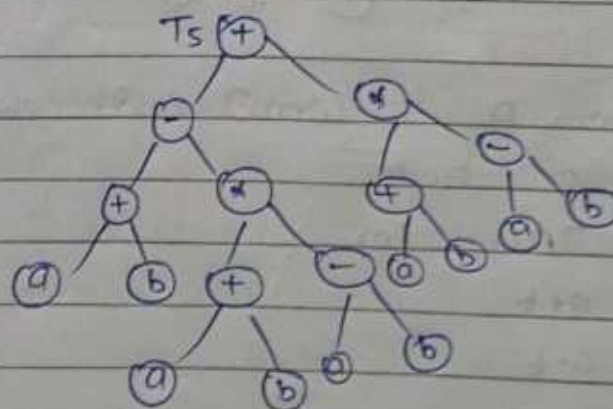
• Step 3: $T_3 = T_1 * T_2$



• Step 4: $T_4 = T_1 - T_3$

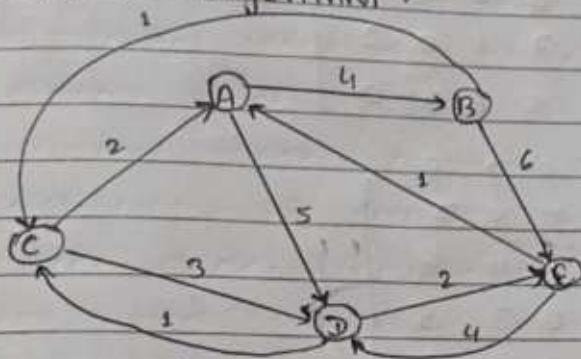


• Step 5: $T_5 = T_4 + T_3$



Section - C

1. Solve the following graph in accordance with the Warshall's algorithm.



$$D^0 = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & \infty & 5 & \infty \\ \infty & 0 & 1 & \infty & 6 \\ 2 & \infty & 0 & 3 & \infty \\ \infty & \infty & 1 & 0 & 2 \\ 1 & \infty & \infty & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D^1 = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & \infty & 5 & \infty \\ \infty & 0 & 1 & \infty & 6 \\ 2 & 6 & 0 & 3 & \infty \\ \infty & \infty & 1 & 0 & 2 \\ 1 & 5 & \infty & 4 & 0 \end{bmatrix} \end{matrix}$$

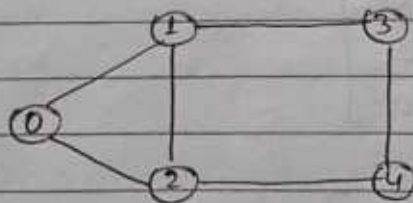
$$D^2 = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 10 \\ \infty & 0 & 1 & \infty & 6 \\ 2 & 6 & 0 & 3 & 12 \\ \infty & \infty & 1 & 0 & 2 \\ 1 & 5 & 6 & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D3 = \begin{matrix} & A & B & C & D & E \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 10 \\ 3 & 0 & 1 & 4 & 6 \\ 2 & 6 & 0 & 3 & 12 \\ 3 & 7 & 1 & 0 & 2 \\ 1 & 5 & 6 & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D4 = \begin{matrix} & A & B & C & D & E \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 7 \\ 3 & 0 & 1 & 4 & 6 \\ 2 & 6 & 0 & 3 & 5 \\ 3 & 7 & 1 & 0 & 2 \\ 1 & 5 & 5 & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D5 = \begin{matrix} & A & B & C & D & E \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 7 \\ 3 & 0 & 1 & 4 & 6 \\ 2 & 6 & 0 & 3 & 5 \\ 3 & 7 & 1 & 0 & 2 \\ 1 & 5 & 5 & 4 & 0 \end{bmatrix} \end{matrix}$$

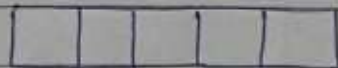
2. Solve the following graph with DFS with BFS.



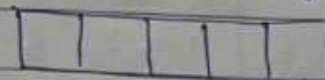
Ans

BFS : (Breath First search)

STEP 1: initially Queue and visited array are empty.

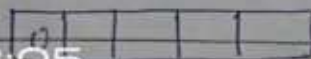


visited



Queue

STEP 2: push 0 to Queue and mark it visited.

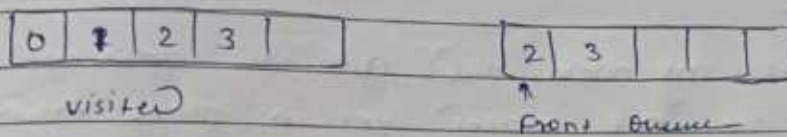


Front Queue

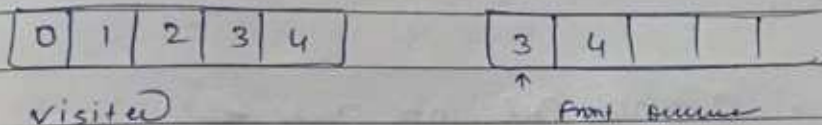
Step 3: Remove 0 from front of queue and visit the unvisited neighbours and push them into queue.



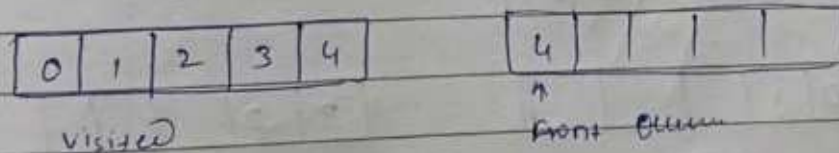
Step 4: Remove 1 from front and visit unvisited neighbours and push them in queue.



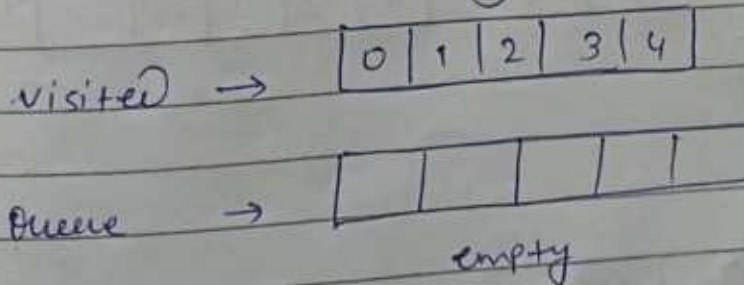
Step 5: Remove Node 2 and push into queue.



Step 6: Remove node 3 and push into queue. neighbours of node 3 is visited.



Step 7: Remove node 4 & push into queue.
 node 4 is already in final state.



- DFS (Depth First Search):-

STEP 1: initially stack and visited arrays are empty-



stack



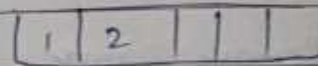
visited

visit any node and put its adjacent nodes which are not visited yet in stack then visit to node which is at top of stack and pop it

STEP 2: visit 0 and put into stack

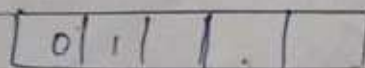


visited

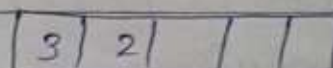


stack

STEP 3:

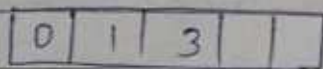


visited



stack

STEP 4:

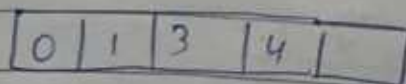


visited



stack

STEP 5:



visited



stack

step 6

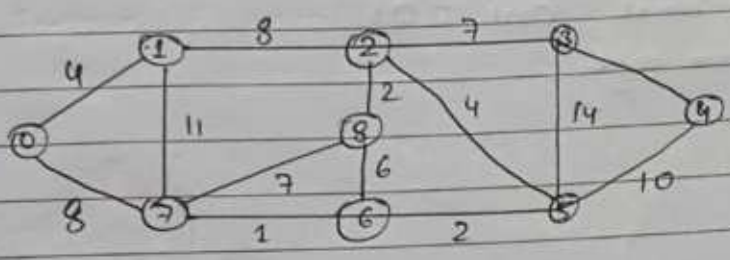
0 | 1 | 3 | 4 | 2

visited

Stack

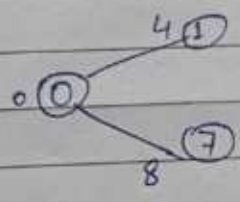
3. Find out minimum cost spanning tree using dijkstra algorithm.

Ans

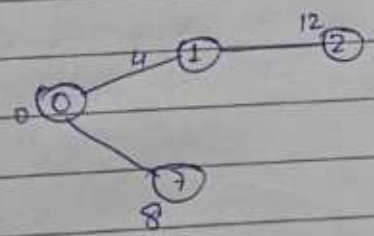


Ans

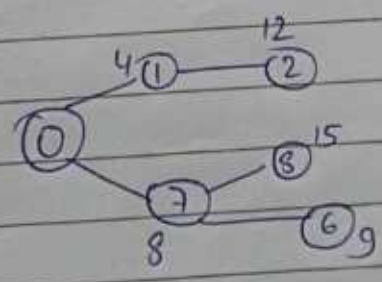
step 1



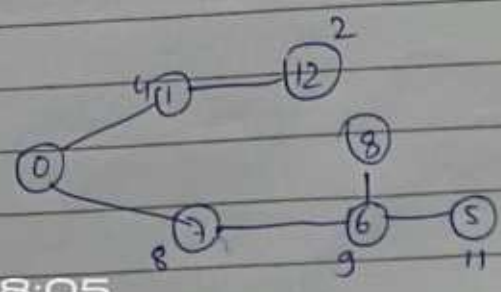
step 2

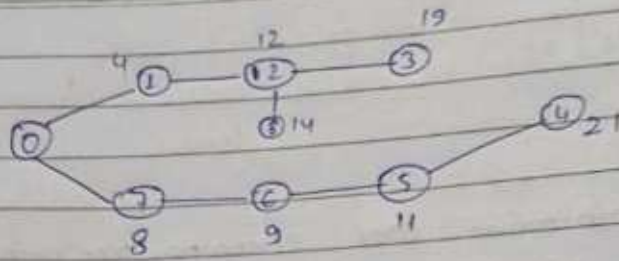


step 3



step 4



Steps

Total Minimal Cost = 21
from source node

The end

~~Suby~~
2/01/2024

Good